Claude Code
Best Practice Guide

Command · Skill · Agent · Orchestration

Source: github.com/shanraisshan/claude-code-best-practice
Tips from Boris Cherny, Thariq Shihipar & Claude Code Team
Based on Anthropic Official Documentation

March 2026 · v2.1+

Table of Contents

1. Core Concepts
- 1.1 Command
- 1.2 Skill
- 1.3 Agent (Sub-Agent)
- 1.4 Comparison Table

2. Hierarchy & Decision Flow
- 2.1 Daily Usage Distribution
- 2.2 Decision Tree

3. Orchestration Workflow
- 3.1 Command → Agent → Skill Pattern
- 3.2 Working Example

4. Memory (CLAUDE.md) Management
- 4.1 Loading Mechanism
- 4.2 Monorepo Strategy

5. MCP Server Configuration

6. Settings & Configuration

7. Hooks

8. Agent Memory (Persistent)

9. Golden Rules & Tips
- 9.1 Workflow Tips
- 9.2 Context Management
- 9.3 Debugging
- 9.4 Tools & Utilities

10. Do's and Don'ts

1. Core Concepts

Claude Code CLI is built on three fundamental building blocks. Each has a different purpose, execution model, and context cost.

1.1 Command

Definition: Entry-point prompt templates triggered by typing `/command-name`. Injected into the current context — does NOT open a new isolated context.

Location: `.claude/commands/<name>.md`

Frontmatter Fields

Field	Type	Required	Description
`description`	string	Recommended	What the command does. Shown in autocomplete
`argument-hint`	string	No	Hint shown during autocomplete (e.g., `[issue-number]`)
`allowed-tools`	string	No	Tools allowed without permission prompts
`model`	string	No	Model to use: `haiku`, `sonnet`, `opus`

Dynamic Variables

Variable	Description
`$ARGUMENTS`	All arguments passed to the command
`$0`, `$1`, `$2`	0-indexed argument access
`${CLAUDE_SESSION_ID}`	Current session identifier
`!`shell-command``	Injects shell output into prompt (evaluated at runtime)

Example: Minimal Command

```
---
description: Build firmware and run tests
model: haiku
---

Build the project firmware and run all unit tests.
Summarize the results.
```

Example: Full-Featured Command

```
---
description: Fix a GitHub issue by number
argument-hint: [issue-number]
allowed-tools: Read, Edit, Write, Bash(gh *), Bash(npm test *)
model: sonnet
---

Fix GitHub issue $0 following our coding standards.

## Context
- PR diff: !`gh pr diff`
- Issue details: !`gh issue view $0`

## Steps
1. Read the issue description
2. Understand the requirements
3. Implement the fix
4. Write tests
5. Create a commit

Session: ${CLAUDE_SESSION_ID}
```

Priority Order

Location	Scope	Priority
`.claude/commands/`	This project only	1 (Highest)
`~/.claude/commands/`	All your projects	2
`<plugin>/commands/`	Where plugin is enabled	3 (Lowest)

1.2 Skill

Definition: Reusable knowledge packages and workflows. Injected into the current context on demand (progressive disclosure).

Location: `.claude/skills/<name>/SKILL.md`

Frontmatter Fields

Field	Description
`name`	Display name and /slash-command. Defaults to directory name if omitted
`description`	What the skill does. Used for autocomplete and auto-discovery
`user-invocable`	Set to `false` to hide from / menu — background knowledge for agent preloading
`allowed-tools`	Tools allowed without permission when skill is active

Two Skill Patterns

PATTERN 1: AGENT SKILL (PRELOADED)
Defined in the agent's `skills:` field. Auto-injected into context when agent starts. This is the agent's built-in domain knowledge.

PATTERN 2: STANDALONE SKILL (ON-DEMAND)
Invoked via `/skill-name` or `Skill()` tool. Runs independently, not tied to any agent. Loaded only when needed.

Example: Standalone Skill

```
---
name: svg-card-creator
description: >
  Creates SVG visual cards from data. Use when report
  or dashboard output is needed.
user-invocable: true
---

# SVG Card Creator

## Rules
- Dimensions: 800x400px, use viewBox
- Fonts: system fonts, don't embed
- Colors: define via CSS variables
- Output: write to output/ directory

## Template Structure
- Header: top 60px, dark background
- Content: middle area, grid layout
- Footer: date and version
```

Example: Agent Skill (Preloaded — Hidden)

```
---
name: api-conventions
description: API endpoint coding conventions
user-invocable: false
---

# API Conventions
- REST endpoint: /api/v1/{resource}
- Error format: { error: { code, message } }
- Pagination: cursor-based
- Auth: Bearer token in header
```

1.3 Agent (Sub-Agent)

Definition: Autonomous actor running in its own isolated context. Has its own tools, permissions, model, memory, and identity.

Location: `.claude/agents/<name>.md`

Frontmatter Fields

Field	Type	Description
`name`	string	Display name
`description`	string	When to invoke. Use `"PROACTIVELY"` for auto-invocation
`tools`	string	Allowed tools: `Read, Write, Edit, Bash`. Inherits all if omitted
`disallowedTools`	string	Tools to deny
`model`	string	`haiku`, `sonnet`, `opus`, `inherit`
`permissionMode`	string	`acceptEdits`, `plan`, `bypassPermissions`
`maxTurns`	int	Maximum agentic turns before stop
`skills`	list	Skills to preload into agent context at startup
`mcpServers`	list	MCP servers scoped to this agent
`hooks`	object	Lifecycle hooks (PreToolUse, PostToolUse, Stop)
`memory`	string	Persistent memory: `user`, `project`, `local`
`background`	bool	`true` → always runs as background task
`color`	string	CLI output color (`green`, `magenta` etc.)

Example: Code Reviewer Agent

```
---
name: code-reviewer
description: >
  Reviews code for quality, security, and maintainability.
  Can be invoked PROACTIVELY.
tools: Read, Write, Edit, Bash
model: sonnet
memory: user
color: green
---

You are a code reviewer. As you review code, save patterns,
conventions, and recurring issues to your agent memory.

## Review Checklist
1. Security: Hardcoded secrets, injection, XSS
2. Performance: N+1 queries, unnecessary computation
3. Testing: Adequate coverage, edge cases
4. Style: Compliance with project conventions
```

⚠ CRITICAL RULE: AGENT COMMUNICATION

Agents **cannot** invoke other agents via bash commands. Use the `Agent()` tool instead:

```
Agent(
  subagent_type="code-reviewer",
  description="Review the code",
  prompt="Review the last commit for security issues",
  model="sonnet"
)
```

Avoid vague terms like `"launch"` or `"run"` — Claude may interpret them as bash commands.

1.4 Comparison Table

Feature	Command	Skill	Agent
Context	Injected into current	Injected into current	**New isolated context**
Trigger	`/command-name`	`/skill-name` or preload	`Agent()` tool
Purpose	Orchestration, user interaction	Knowledge / know-how	Independent task execution
Context cost	Low	Medium	**High**
Own model	Yes	No	Yes
Persistent memory	No	No	**Yes**
Tool restriction	allowed-tools	No	tools / disallowedTools
When to use	Workflow entry point	Reusable domain knowledge	Isolated, heavy tasks

2. Hierarchy & Decision Flow

2.1 Daily Usage Distribution

80%
Vanilla Claude Code
Good CLAUDE.md + direct prompts

→

15%
Command + Skill
Recurring workflows

→

5%
Agent
Isolated heavy tasks

GOLDEN RULE

"Vanilla Claude Code is better than any workflow for smaller tasks." Don't add unnecessary complexity. Workflows are only for recurring or coordination-heavy work.

2.2 Decision Tree

```
Is the task simple?
  → Yes → Vanilla Claude Code, no setup needed
  → No ↓

Is it a recurring workflow?
  → Yes → Write a COMMAND (/build, /test, /deploy)
  → No ↓

Domain knowledge used in multiple places?
  → Yes → Write a SKILL (protocol-spec, coding-patterns)
  → No ↓

Need isolated work without polluting main context?
  → Yes → Define an AGENT (reviewer, test-runner)
  → No → Vanilla Claude Code
```

3. Orchestration Workflow

3.1 Command → Agent → Skill Pattern

The most powerful usage combines all three building blocks in layers:

Layer	Role	Description
Command	Entry point	Interacts with user, coordinates agents and skills
Agent	Data fetcher	Collects data with preloaded skill, returns result to command
Skill	Output producer	Runs independently, produces output from agent's data

3.2 Working Example: Weather Orchestrator

```
# Flow
User → /weather-orchestrator (Command)
  ├── Step 1: AskUser → Celsius or Fahrenheit?
  ├── Step 2: Agent tool → weather-agent
  │     └── Preloaded Skill: weather-fetcher
  │     └── Fetches temperature from Open-Meteo API
  │     └── Returns: temperature=26, unit=Celsius
  ├── Step 3: Skill tool → weather-svg-creator
  │     ├── Creates: weather.svg
  │     └── Writes: output.md
  └── Output: SVG card + summary file
```

Command File

```
# .claude/commands/weather-orchestrator.md
---
description: Fetch weather data and create an SVG card
model: haiku
---

1. Ask the user: Celsius or Fahrenheit?
2. Invoke weather-agent (to fetch temperature)
3. Pass the result to /weather-svg-creator skill
4. Present the SVG and summary to user
```

Agent File (with Preloaded Skill)

```
# .claude/agents/weather-agent.md
---
name: weather-agent
description: Fetches weather data
tools: Bash(curl *)
model: haiku
skills:
  - weather-fetcher   # Preloaded at startup
---

Follow the preloaded skill instructions to fetch
temperature data and return the result.
```

Core Principles

- **Command** = orchestrator (user interaction + coordination)
- **Agent** = data fetching (with preloaded skill)
- **Skill** = independent output production
- Each component has a **single responsibility**: Fetch (agent) → Render (skill)

4. Memory (CLAUDE.md) Management

4.1 Loading Mechanism

Type	Loading	Description
Ancestor (parent dirs)	Auto at startup	Claude walks UP and loads all CLAUDE.md files found
Descendant (child dirs)	Lazy	Only loaded when interacting with files in that directory
Sibling (peer dirs)	**NEVER**	Working in frontend/ will NOT load backend/CLAUDE.md
Global	Always	`~/.claude/CLAUDE.md` — applies to all sessions

4.2 Monorepo Strategy

```
/project-root/
├── CLAUDE.md # Root — shared rules (always loaded)
├── frontend/
│   └── CLAUDE.md # Frontend rules (lazy)
├── backend/
│   └── CLAUDE.md # Backend rules (lazy)
├── firmware/
│   └── CLAUDE.md # Firmware rules (lazy)
└── api/
    └── CLAUDE.md # API rules (lazy)
```

CLAUDE.MD RULES

- Keep each file **under 200 lines** (ideal: 60 lines)
- Use `CLAUDE.local.md` for personal preferences (add to `.gitignore`)
- `/memory` and `/rules` **guarantee nothing** — even writing MUST in all caps doesn't ensure compliance
- Instead of stuffing CLAUDE.md, **split knowledge into skills**

5. MCP Server Configuration

MCP (Model Context Protocol) servers connect Claude Code to external tools, databases, and APIs.

Configuration Locations & Priority

Location	Scope	Priority
Agent frontmatter (`mcpServers`)	Scoped to that agent	1 (Highest)
`.mcp.json` (repo root)	Project	2
`~/.claude.json`	User (all projects)	3

Recommended MCP Servers

Server	Package	Purpose
Context7	`@upstash/context7-mcp`	Fetches up-to-date library docs, prevents hallucinated APIs
Playwright	`@playwright/mcp`	Browser automation — UI testing, screenshots, forms
Chrome DevTools	—	Connects to real Chrome — console, network, DOM debugging
DeepWiki	`deepwiki-mcp`	Wiki-style documentation for any GitHub repo
Excalidraw	—	Architecture diagrams, flowcharts as hand-drawn sketches

Example .mcp.json

```
{
  "mcpServers": {
    "context7": {
      "command": "npx",
      "args": ["-y", "@upstash/context7-mcp"]
    },
    "playwright": {
      "command": "npx",
      "args": ["-y", "@playwright/mcp"]
    }
  }
}
```

WARNING: MORE MCP ≠ BETTER

"Went overboard with 15 MCP servers thinking more = better. Ended up using only 4 daily." — r/mcp (682 upvotes). Only add what you actually use.

6. Settings & Configuration

File Hierarchy

File	Scope	Description
`~/.claude/settings.json`	Global	User settings applied to all projects
`.claude/settings.json`	Project	Team-shared project settings
`.claude/settings.local.json`	Personal	git-ignored personal overrides
`managed-settings.json`	Organization	Cannot be overridden, highest precedence

Recommended Settings

- **Thinking mode:** `true` — see Claude's reasoning process
- **Output Style:** `Explanatory` — detailed output with ★ Insight boxes (for new codebases)
- **Output Style:** `Learning` — coaching mode (for learning)
- Check `settings.json` into **git** — so your team benefits
- Use the `env` field — avoids wrapper scripts (84 environment variables available)

7. Hooks

Deterministic scripts that run **outside** the agentic loop on specific lifecycle events.

Hook Events

Event	When It Fires
`PreToolUse`	Just before a tool is called
`PostToolUse`	After a tool call completes
`UserPromptSubmit`	When user submits a prompt
`Stop`	When Claude stops
`SubagentStart / SubagentStop`	When an agent starts / stops
`Notification`	On notification
`PreCompact`	Before compact operation
`SessionStart / SessionEnd`	Session open / close

Use Cases

- Play a sound on git commit
- Route permission requests to Slack
- Log tool calls for audit
- Nudge Claude to keep going when it stops
- Run lint after every edit

8. Agent Memory (Persistent)

Introduced in Claude Code v2.1.33. Gives each agent its own persistent memory store — enabling learning across sessions.

Memory Scopes

Scope	Location	Visibility
`user`	`~/.claude/agent-memory/<agent>/`	Only you, all projects
`project`	`.claude/agent-memory/<agent>/`	Entire team, this project
`local`	`.claude/agent-memory.local/<agent>/`	Only you, this project

How It Works

- First **200 lines** of `MEMORY.md` are injected into system prompt at startup

- `Read`, `Write`, `Edit` tools auto-enabled for memory management
- When exceeding 200 lines, agent moves details into topic-specific files
- Skills = static knowledge (at startup) ↔ Memory = dynamic knowledge (built over time)

Example: Agent with Memory

```
---
name: api-developer
tools: Read, Write, Edit, Bash
model: sonnet
memory: project
skills:
  - api-conventions
  - error-handling-patterns
---

Implement API endpoints. Follow conventions from your
preloaded skills. As you work, save architectural decisions
and discovered patterns to your memory.

Before starting, review your memory.
```

```
~/.claude/agent-memory/api-developer/
├── MEMORY.md # Primary (first 200 lines loaded)
├── rest-patterns.md # Topic-specific
└── error-catalog.md # Topic-specific
```

9. Golden Rules & Tips

9.1 Workflow Tips

Rule	Description
Always start with **plan mode**	`/plan` command — read first, then implement
Use `ultrathink`	Add "ultrathink" to prompts → high effort reasoning
Feature-specific agents	"auth-module-reviewer" instead of generic "QA engineer"
Progressive disclosure with skills	Don't cram everything into CLAUDE.md — split into skills
Phase-gated plans	Each phase should have unit + integration + automation tests
Cross-model review	Have a different model review your plan (e.g., Codex)
Let Claude interview you	Use `AskUserQuestion` tool to create specs, then execute in new session
Write detailed specs	Reduce ambiguity — the more specific, the better the output

9.2 Context Management

Rule	Description
/compact at 50%	Avoid "agent dumb zone" — compress before context fills
`/clear` to reset	Reset session when switching tasks
`/context` to monitor	View context usage as a colored grid
`Esc Esc` or `/rewind`	If Claude goes off track, rewind instead of fixing in same context
Say `"use subagents"`	Offload heavy work to subagent, keep main context clean
Name your sessions	`/rename [TODO - refactor]` → later `/resume`

9.3 Debugging

Method	Description
`/doctor`	Check Claude Code installation health
Run as background task	Open terminal with logs as background process
MCP browser debug	Claude in Chrome / Playwright / Chrome DevTools MCP
Provide screenshots	Share screen captures for visual issues
Cross-model QA	Review implementation with a different model/tool
Challenge Claude	"Grill me on these changes, don't make a PR until I pass your test"

9.4 Tools & Utilities

Tool	Description
iTerm / Ghostty / tmux	Use terminal instead of IDE
Wispr Flow	Voice prompting — claimed 10x productivity
Wildcard permissions	`Bash(npm run *)`, `Edit(/docs/**)` — instead of dangerously-skip
`/sandbox`	File and network isolation to reduce permission prompts
Agent Teams + tmux	Multiple agents for parallel development
Git Worktrees	Isolated branch per agent — parallel work
Ralph Wiggum plugin	Autonomous loop for long-running tasks

10. Do's and Don'ts

✅ DO	❌ DON'T
Keep CLAUDE.md under 200 lines	Cram everything into one CLAUDE.md
Use layered CLAUDE.md in monorepos	Manage with a single massive CLAUDE.md
Define feature-specific agents	Use generic "QA engineer", "backend dev" agents
Use commands for workflows	Open an agent for simple workflows
Commit as soon as task completes	Work for long stretches without committing
`/compact` at 50%	Wait until context is full
`Esc Esc` / `/rewind` to undo	Try to fix mistakes in the same context
Use wildcard permissions	Use `dangerously-skip-permissions`
Update Claude Code daily	Stay stuck on an old version
Start with plan mode	Dive straight into implementation
Write detailed specs, reduce ambiguity	Expect quality output from vague prompts
Check `settings.json` into git	Work without sharing team settings
Invoke agents with `Agent()` tool	Call agents via bash commands
Only add MCP servers you actually use	Install 15 MCPs thinking "more = better"
Prefer vanilla CC for small tasks	Set up workflows/agents for everything

Source: shanraisshan/claude-code-best-practice · Boris Cherny & Claude Code Team · Anthropic Docs
This document is a reference guide. Claude Code is continuously updated — follow the official changelog.